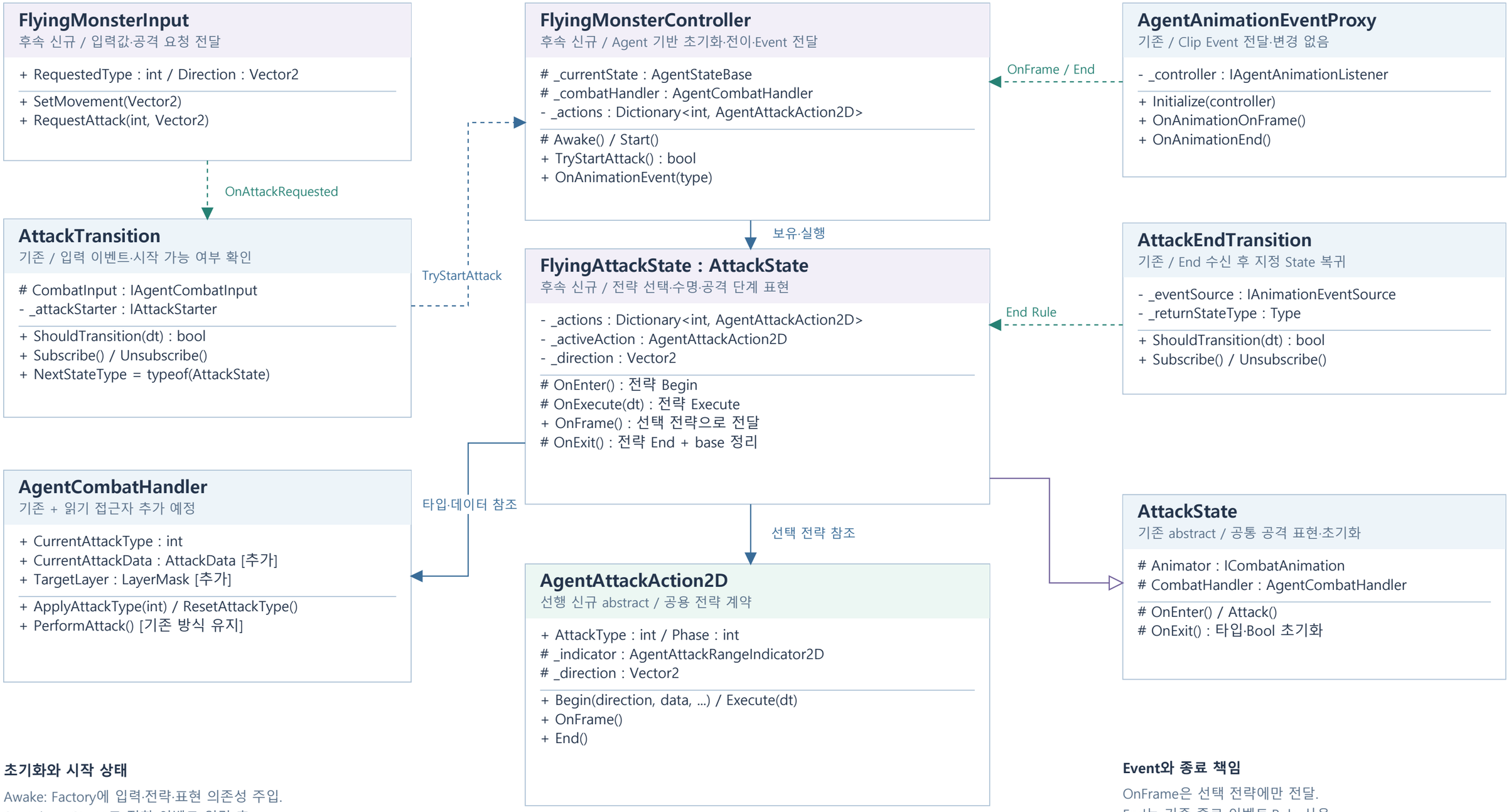
공격 FSM과 전략 선택의 관계

기존 FSM 계약을 유지하고 FlyingAttackState가 공용 전략을 사용한다. 파란 상자: 기존 / 연두: 선행 신규 / 보라: 후속 신규



초기화와 시작 상태

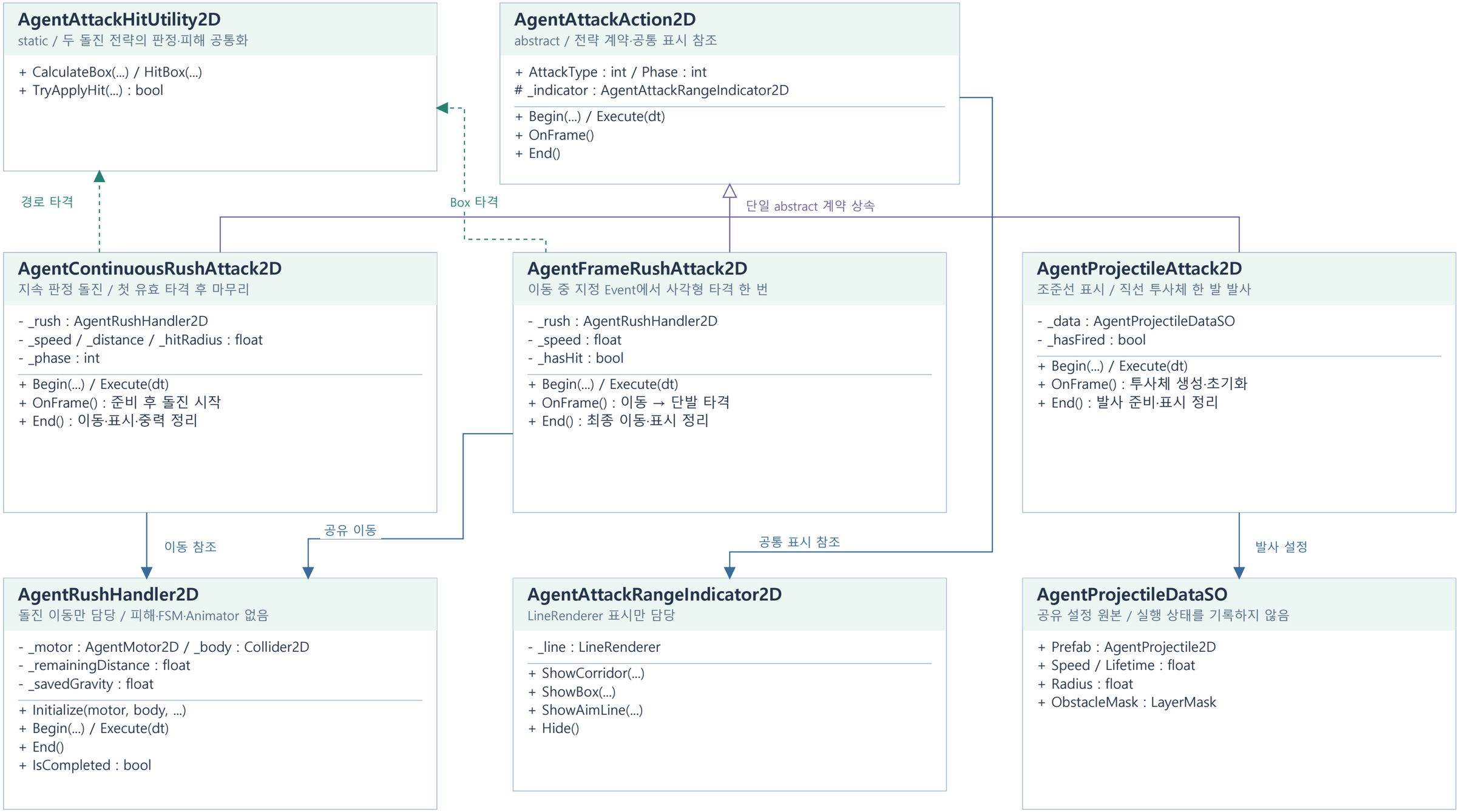
Awake: Factory에 입력·전략·표현 의존성 주입.
Start: base.Start()로 전환 이벤트 연결 후
구체 Controller에서 ChangeState(Flight) 호출.
Dictionary 키는 typeof(AttackState) 유지.

Event와 종료 책임

OnFrame은 선택 전략에만 전달.
End는 기존 종료 이벤트·Rule 사용.
State.Exit에서 전략을 정리하며
전략은 ChangeState·Animator를 호출하지 않음.

공용 공격 전략과 실행 의존성

공용 공격 관련 클래스 9개 중 전략·돌진·표시·설정 관계. 모든 연두 상자는 선행 계획에서 제작할 예정



전략별로 다른 종료 시점

Attack1: 첫 타격·벽·거리 종료 후 마무리 모션. Attack2: 벽에서는 이동만 정지, 최종 End까지 모션 유지. Attack3: 발사 후 투사체는 독립 실행.

투사체 수명과 피해·범위 표시의 공통화

공용 Utility에서 방향 지정 Box 계산·타격 전달을 공유한다. 기존 CombatHandler에는 읽기 접근자만 추가하고 방향 overload는 넣지 않음

AgentProjectileAttack2D
선행 신규 / 생성 시 실행 데이터 전달

- _data : AgentProjectileDataSO
- _hasFired : bool

+ OnFrame() : 한 발 생성
+ End() : 이미 발사한 개체는 유지

생성·주입

AgentProjectile2D
선행 신규 / 발사 후 독립 이동·충돌·소멸

- _data : AgentProjectileDataSO
- _damage : float / _direction : Vector2
- _remainingLife : float
- _owner : GameObject / _targetLayer : LayerMask
- _isImpacted : bool

+ Initialize(data, damage, direction, ...)
- FixedUpdate() / BeginImpact()
+ OnAnimationEnd() : 최종 제거

설정 참조

AgentProjectileDataSO
선행 신규 / 공유 설정 참조

+ Prefab : AgentProjectile2D
+ Speed / Lifetime / Radius : float
+ ObstacleMask : LayerMask

State 경유: 데이터·Layer

첫 충돌 피해

AgentCombatHandler
기존 + 전략에서 읽을 접근자 추가

+ CurrentAttackType : int
+ CurrentAttackData : AttackData [추가]
+ TargetLayer : LayerMask [추가]

+ PerformAttack() : 기존 사각형 방식

타격 공유

AgentAttackHitUtility2D
선행 신규 static / 범위·타격 전달 공통화

+ CalculateBox(origin, direction, data, ...)
+ HitBox(center, size, angle, ...)
+ TryApplyHit(target, damage, origin) : bool

피해

IDamageable
기존 interface / 피해 수신

+ TakeDamage(float damage)

넉백

IKnockbackListener
기존 interface / 넉백 수신

+ HandleKnockback(Vector2 direction)

전략 경유: Box 결과

AgentAttackRangeIndicator2D
선행 신규 / 계산된 좌표만 표시

- _line : LineRenderer

+ ShowBox(center, size, angle)
+ ShowCorridor(...) / ShowAimLine(...)
+ Hide()

AgentRushHandler2D
선행 신규 / 실제 변위·벽·중력 복구

- _motor : AgentMotor2D
- _body : Collider2D / _savedGravity : float

+ Begin(...)
+ Execute(dt)
+ End()

물리 적용

AgentMotor2D
기존 + 범용 XY 속도·정지 추가

- _rb : Rigidbody2D

+ Move(input, speed) / Jump(force)
+ StopHorizontal()
+ Move2D(input, speed) [추가]
+ Stop() [추가]

실행 데이터와 표시의 수명

피해·방향은 발사 시 복사하고 SO는 수정하지 않는다. FrameRush의 표시와 타격은 같은 Box 결과를 사용한다. 이미 발사한 투사체는 Actor의 피격·사망과 독립적이다.